

The complexity of envy-free graph cutting^{*}

Argyrios Deligkas^{ID a,*}, Eduard Eiben^{ID a}, Robert Ganian^{ID b}, Thekla Hamm^c,
Sebastian Ordyniak^{ID d}

^a Royal Holloway University of London, UK

^b TU Wien, Austria

^c Eindhoven University of Technology, Netherlands

^d University of Leeds, UK

ARTICLE INFO

Keywords:

Fair division
Graph cutting
Envy-freeness

ABSTRACT

We consider the problem of fairly dividing a set of heterogeneous divisible resources among a set of agents with different preferences. We focus on the case where the resources correspond to the edges of a connected graph, every agent must be assigned a connected piece of this graph, and the fairness notion considered is the classical envy-freeness. The problem is NP-complete, and we analyze its complexity with respect to two natural complexity measures: the number of agents, and the number of edges in the graph. While the problem remains NP-hard even for instances with 2 agents, we provide a dichotomy characterizing the complexity of the problem when the number of agents is constant based on structural properties of the input graph. For the latter case, we design a polynomial-time algorithm for the problem when the graph has a constant number of edges.

1. Introduction

Cake cutting is, without a doubt, among the most influential problems in social choice and has received significant attention in computer science, mathematics, and economics [1–4]. The cake corresponds to a heterogeneous divisible resource that is to be divided between a set of n agents with different preferences in a “fair” manner. In this paper, the fairness concept we focus on is *envy-freeness*, where every agent prefers the piece of the cake they get allocated over the piece any other agent gets.

In the classical formulation of the problem, the cake is represented as an interval and the preference of every agent over the cake is given via a *valuation* over any subinterval of the cake. In this setting, Dubins and Spanier [5] showed that an envy-free allocation *always* exists for arbitrary valuations for the agents, where the piece an agent gets consists of a countable number of subintervals. Stromquist [6] strengthened this result by removing the possibility of pieces that consist of a “union of crumbs”. In fact, he showed that there is an envy-free allocation where the piece of every agent is *contiguous*, i.e., it is a *single* subinterval.

Recently, Bei et al. [7] considered a generalized version of cake cutting on *graphs*. This augmented model allows to capture more general scenarios which cannot be represented by splitting an interval into connected pieces—consider, e.g., the task of splitting road or railway networks between companies. We note that these settings do not always give rise to large graphs: for instance, the ICE train network in Germany can be modeled as a graph with merely 23 edges. For another example, suppose that one wants to assign

^{*} A preliminary version of the paper under the same title was presented at IJCAI 2022 [52].

^{*} corresponding author.

E-mail addresses: argyrios.deligkas@rhul.ac.uk (A. Deligkas), eduard.eiben@rhul.ac.uk (E. Eiben), rganian@gmail.com (R. Ganian), thekla.hamm@gmail.com (T. Hamm), sordyniak@gmail.com (S. Ordyniak).

<https://doi.org/10.1016/j.artint.2026.104585>

Received 28 October 2024; Received in revised form 9 June 2026; Accepted 10 June 2026

Available online 16 June 2026

0004-3702/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Table 1

Complexity of EF-GC and EF-VDGC for a constant number of agents for different restrictions on the treewidth (tw) and the maximum degree (Δ). All NP-completeness (NP-c) results hold already for only 2 agents.

tw	Δ	EF-GC	EF-VDGC
2	3	NP-c (Theorem 5)	NP-c (Theorem 5)
2	2	P (Theorem 9)	P (Theorem 9)
1	arbitrary	NP-c (Theorem 3)	P (Theorem 7)
1	constant	P (Theorem 8)	P (Theorem 7)

cleaning and maintenance in a building to a set of individuals or small teams, each with a specialization for certain kinds of areas. This setting, too, can be modeled using a graph that captures the layout of the building. In these as well as other examples, it is still desirable to ensure that each agent receives a connected piece of the graph, but the natural model for the cake is a graph where each individual edge may be split and behaves as a single, uniform piece.

Observe that depending on the setting, it may either be the case that a vertex is allocated to only a single agent (as in the case of junctions in the division of road networks over maintenance companies), or that a vertex merely acts as a bridge between edges and may be used by multiple agents (as in the case of train stations in division of railway networks over rail companies). We call the latter setting “*graph cutting*” and the former “*vertex-disjoint graph cutting*”.

While both fair division problems always admit a contiguous envy-free solution when the underlying graph is a path (since they are special cases of the classical cake cutting problem), this is no longer true for more general graphs. In fact, it is not hard to construct graphs with no envy-free solutions; this holds even for stars with three leaves and two agents with identical valuations. Hence, in the setting studied here, the natural task is to *decide if a solution exists, and if it does to efficiently compute one*.

Our Contributions. In this work, we explore the frontiers of tractability for both variants of graph cutting under the envy-free solution concept; we refer to these problems as EF-GC and EF-VDGC, respectively. While it is not difficult to show that both problems are NP-complete in general, here we analyze the problem with respect to two of the most natural complexity measures that characterize the input: the number of agents and the number of edges.

When considering the number of agents (i.e., n), we begin by extending the NP-completeness lower bounds for both variants to the case of $n = 2$ even on very simple graphs—as simple as two vertices plus a matching (Theorem 4). However, here we can show that the two variants do sometimes behave differently: while EF-GC is also NP-hard on stars when $n = 2$ (Theorem 3), we design a polynomial-time algorithm for EF-VDGC on trees when n is upper-bounded by an arbitrary but fixed constant (Theorem 7). In order to achieve tractability for EF-GC, we need to restrict ourselves to instances with a constant number of agents and where the graph is a tree with a constant maximum degree (Theorem 8).

In fact, we prove this is the best one can do from this perspective. Both problems become NP-hard on instances with two agents and graphs of maximum degree 3 which are “almost trees”—in particular, have treewidth 2 (Theorem 5). On the other hand, we show that both problems are polynomial-time solvable on cycles, which are graphs of maximum degree and treewidth 2 (Theorem 9); this provides a complete dichotomy for the complexity of both problems with respect to treewidth and maximum degree (see Table 1).

Next, we target instances where the number of edges is bounded by a constant. As the main technical contribution of this article, we show that both problems under consideration become polynomial-time tractable under this restriction (Theorem 10). The algorithm is non-trivial and combines insights into the structure of a hypothetical solution with branching techniques, linear programming subroutines and insights from multidimensional geometry.

Related Work. Bei and Suksompong [7] studied graph cutting under the fairness notions of proportionality and equitability; this was the first paper that considered a graph structure with *divisible* items. For indivisible items there are several different graph-based approaches. In the most common modeling scenario the items correspond to the vertices of the graph and each agent must get a connected subgraph [8–14]. One recent closely related work is that of Igarashi and Zwicker [15], which studied the relationship between the notion of graph tangles and the existence of solutions for vertex-disjoint graph cutting regardless of the valuation (or under some very mild assumptions on valuations). Their work can be seen as complementary to ours: while we approach the problem from a computational perspective that takes into account both the valuations and graphs, their techniques allow one to immediately identify graphs whose structure is guaranteed to lead to YES-instances.

A different line of work uses graphs to denote the relationships between the agents, where an agent compares their bundle only against the bundles of the agents they are connected with [16–21].

There is a long line of papers for classical cake cutting and its extensions. Su [22] gave an alternative proof of existence of contiguous envy-free solutions using Sperner’s Lemma; this proof was used in [23] to place the approximate version of the computational problem in PPA. Recently, Segal-Halevi and Suksompong [24] extended the existential result for groups of agents and [25,26] extended the result for multilayered cakes. Polynomial-time algorithms that compute approximately envy-free contiguous allocations were presented in [23,27,28], while [29] provides a polynomial-time algorithm for the problem for a very restricted class of valuations; to the best of our knowledge this is the only positive result for exact solutions. On the other hand, non-contiguous envy-free cake cutting is easy to solve [30–32].

In addition to the above, there are many other works that study variants of cake cutting [33–43]. In this context, we emphasize that our work focuses on the case where the utility of each edge is uniformly distributed, while some previous works on graph cutting considered the intra-edge utility to itself be heterogeneous. We phrase our results in the setting of the *direct revelation* model, where

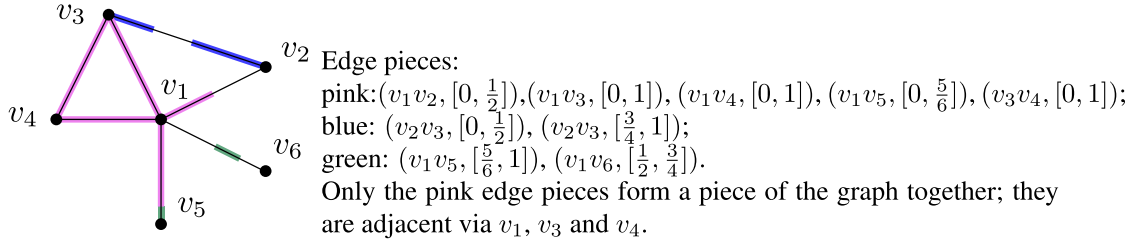


Fig. 1. Examples of pieces of edges and of a graph.

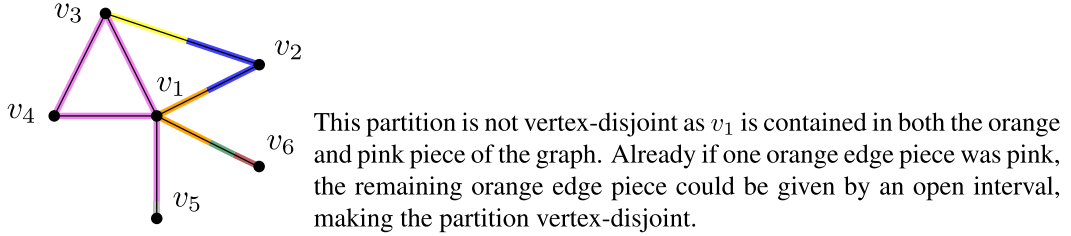


Fig. 2. Non-vertex-disjoint partition of a graph into seven connected pieces of different colors.

each agent reveals its entire utility function to the algorithm, while some of the related works study the *query model* in which the utility functions are only accessible via queries. However, this latter distinction is merely for convenience: since the edge utilities are uniformly distributed, one can recover the exact utility functions of all agents via a polynomial number of queries.

2. Preliminaries and problem definition

Notation. For rational numbers $i, j \in \mathbb{Q}$, we use the standard notation $[i, j] = \{k \in \mathbb{Q} \mid i \leq k \leq j\}$ for intervals and for an interval $I = [i, j]$ (or $I = (i, j)$ if it is open) we denote its length by $|I| = \max\{j - i, 0\}$. We denote the set of all non-negative rational numbers by $\mathbb{Q}^+$. Further, we refer to the handbook by [44] for standard graph terminology. Throughout the paper we will consider simple undirected graphs.

Graph Cutting. Consider a set A of n agents, a connected graph $G = (V, E)$, and for each agent $a \in A$ an edge weight function $u_a : E \rightarrow \mathbb{Q}^+$ over the edges of G ; u_a is the *utility* (function) of agent a , and for a specific edge $e \in E$ we call $u_a(e)$ the *utility* of a for e .

A *piece of an edge* e is a tuple (e, I) where $I \subseteq [0, 1]$ is a (possibly empty, closed, open or half-open) interval. We assume an arbitrary but fixed order V of the vertices of G , and say that pieces (e, I) and (e', I') of two different edges $e, e' \in E$ are *adjacent* if

- there is some $v \in V$ such that $e = uv$ and $e' = vv'$ (i.e. e and e' are adjacent in G), and
- if u has a smaller index in the ordering of V than v , then $1 \in I$ and $0 \in I'$ otherwise, and
- similarly, if u' has a smaller index in the ordering of V than v , then $1 \in I'$ and $0 \in I$ otherwise.

A *piece of G* is a collection P of pieces of edges of e such that for every pair of pieces of edges $(e_0, I_0), (e_\ell, I_\ell)$ in P there is a sequence of pieces of edges $(e_1, I_1), \dots, (e_{\ell-1}, I_{\ell-1})$ in P such that for all j with $0 \leq j < \ell$, (e_j, I_j) and (e_{j+1}, I_{j+1}) are adjacent. An example is provided in Fig. 1.

The utility of an agent a for a piece P is given as $u_a(P) = \sum_{(e, I) \in P} |I| \cdot u_a(e)$. Note that $\{(e, [0, 1]) \mid e \in E\}$ is also a piece of G . As is standard, our algorithms will assume *normalized* utilities, i.e., $u_a(G) = 1$ for all $a \in A$.¹

A *partition* of G into pieces is a set Π of pieces such that for every edge $e \in E$ and every real $0 \leq \alpha \leq 1$, there is precisely one piece $P \in \Pi$ and one $(e, I) \in P$ such that $\alpha \in I$. In some cases it is also necessary to allocate each vertex to a single piece: a partition of G into pieces is *vertex-disjoint* if all pieces of edges containing a vertex belong to the same piece of the graph. See Fig. 2 for an example.

Finally we are ready to define our problem of interest. In (piecewise) envy-free graph cutting (EF-GC) we are given agents A , graph G and utilities $u_a : E \rightarrow \mathbb{Q}^+$ for each agent $a \in A$. Our task is to construct a partition Π of G into pieces and a bijection (called an *assignment*) $\pi : A \rightarrow \Pi$ such that for every pair of agents $a, a' \in A$, it holds that $u_a(\pi(a)) \geq u_a(\pi(a'))$. This condition is commonly referred to as *envy-freeness* in assignment and allocation problems, and when it is violated for some a and a' , we say that a *envies* a' .

We can analogously define the problem of (piecewise) envy-free vertex-disjoint graph cutting (EF-VDGC) by additionally requiring Π to be vertex-disjoint. In fact, there is a simple reduction from the former problem to EF-VDGC.

Observation 1. *EF-GC can be reduced to EF-VDGC in polynomial time.*

¹ Instances with non-normalized utilities can be trivially transformed into equivalent ones with normalized utilities.

Proof. Given an instance $I = (A, G, \{u_a\}_{a \in A})$ of EF-GC, we construct an instance $I' = (A, G', \{u'_a\}_{a \in A})$ of envy-free vertex-disjoint graph cutting as follows. Let $m = |E(G)|$. Towards constructing G' , we replace each vertex $v \in V(G)$ with a set of vertices $v_1, \dots, v_m$, add “transparent” edges between each pair v_i, v_j (for $1 \leq i < j \leq m$) of such vertices, and then for each edge $e_i = ab$ in $E(G)$ (where $i \in [m]$) we construct a new “solid” edge $a_i b_i \in E(G')$. Intuitively, this amounts to replacing each original vertex v with a clique and redirecting each edge incident to v to its own vertex in said clique. As for $\{u'_a\}_{a \in A}$, we set the valuations of each solid edge in G' to precisely match the valuations of the edge of G that it originated from, while for each transparent edge in G' we set all valuations to 0. This completes the description of the reduction.

Towards establishing correctness, assume that I is a Yes-instance as witnessed by some envy-free assignment Π . We can translate this assignment into a corresponding vertex-disjoint envy-free assignment Π' of I' by replicating Π on the solid edges of G' , and then ensuring connectivity by having agent a also take the transparent edges between each v_i and v_j such that $v \in V(G)$, $1 \leq i < j \leq m$, and where Π assigns the v -endpoint of both e_i and e_j to a . Note that this leaves some transparent edges unassigned, but since these have an utility of 0 for every agent we may simply assign the leftover transparent edges to any agent while preserving connectivity.

For the converse, assume that Π' is a vertex-disjoint envy-free assignment in I' . To construct an envy-free assignment Π in I , it suffices to replicate Π' on the solid edges while ignoring the behavior of Π' on transparent edges. $\square$

Bounding the Number of Cells in Metric Spaces. One prominent tool our main algorithm for solving EF-GC and EF-VDGC uses is a theorem that applies to the behavior of polynomials in higher-dimensional spaces. To provide a high-level intuition, consider a d -dimensional space that is cut into regions by s -many hyperplanes or, more generally, “well-behaved cuts” defined by bounded-degree polynomials. The combination of these cuts splits the whole space into “cells”, each consisting of points that lie on the same side of each of the s -many cuts. While the trivial bound for the number of these cells is 2^s , it can be shown that the number of such cells is in fact polynomial in s for fixed d and that representatives of these cells can be computed efficiently.

Let $\mathcal{V} = \mathbb{R}^d$ and let us denote the ring of polynomials over variables $X_1, \dots, X_d$ with coefficients in $\mathbb{R}$ by $\mathbb{R}[X_1, \dots, X_d]$. For a set of s polynomials $\mathcal{P} = \{P_1, \dots, P_s\} \subset \mathbb{R}[X_1, \dots, X_d]$, a sign condition is specified by a sign vector $\sigma \in \{-1, 0, +1\}^s$, and the sign condition is non-empty over $\mathcal{V}$ with respect to $\mathcal{P}$ if there is a point $x \in \mathcal{V}$ such that $\sigma = (\text{sign}(P_1(x)), \dots, \text{sign}(P_s(x)))$, where $\text{sign}(\alpha)$ is the sign function defined as

$$\text{sign}(\alpha) = \begin{cases} 1, & \text{if } \alpha > 0, \\ 0, & \text{if } \alpha = 0, \\ -1, & \text{if } \alpha < 0. \end{cases}$$

The *realization space* of $\sigma \in \{-1, 0, +1\}^s$ over $\mathcal{V}$ is the set

$$R(\sigma) = \{x | x \in \mathcal{V}, \sigma = (\text{sign}(P_1(x)), \dots, \text{sign}(P_s(x)))\}.$$

If $R(\sigma)$ is not empty then each of its non-empty semi-algebraically connected (which is equivalent to just connected on semi-algebraic sets [45, Theorem 5.22]) components is a *cell* of $\mathcal{P}$ over $\mathcal{V}$.

The following result, sometimes called Warren’s Theorem [46], gives an algorithm to compute a point in each cell of $\mathcal{P}$ over $\mathcal{V}$.

Proposition 1. (Basu, Pollack and Roy [45] (Theorem 13.22), see also Simonov et al. [47]) Let $\mathcal{P} \subset \mathbb{R}[X_1, \dots, X_d]$ be a finite set of s linear polynomials, i.e., each $P \in \mathcal{P}$ is of degree at most 1. Let D be a ring generated by the polynomials in $\mathcal{P}$. There is an algorithm using at most $\mathcal{O}(s^d)$ arithmetic operations in D which, given $\mathcal{P}$, computes a set of points meeting every non-empty cell of $\mathcal{V}$ over $\mathcal{P}$.

We remark Proposition 1 is merely the special case of Basu, Pollack and Roy [45] (Theorem 13.22) restricted to $\mathcal{V} = \mathbb{R}^d$ and linear polynomials.

3. Instances with few agents

In this section we consider the complexity of EF-GC and EF-VDGC for instances with only a few agents. Interestingly, we will show that while both problems are NP-hard even for instances with only two agents, EF-GC turns out to be significantly harder when additional restrictions are considered for the input graph.² In particular, while EF-GC is already NP-hard on stars, EF-VDGC can be solved in polynomial-time (for a fixed number of agents) even on trees and only becomes NP-hard on graphs that have a vertex deletion set into a matching. Moreover, the problem becomes much harder if we relax the graph structure from trees to “tree-like graphs”: both problems become NP-hard on graphs that have treewidth 2 and maximum degree 3 (see Fig. 3 for an illustration).

Theorem 3. EF-GC is NP-hard even when restricted to instances with two agents with identical utility functions where the graph is a star.

Proof. We provide a simple polynomial-time reduction from the well-known NP-complete NUMBER PARTITIONING problem. Let S be an instance of NUMBER PARTITIONING, i.e., S is a multi-set of non-negative integers and the aim is to decide whether there is a partition of S into two subsets S_1 and S_2 such that $\sum_{s \in S_1} s = \sum_{s \in S_2} s$. Without loss of generality, throughout the article we assume that for each integer $s \in S$, $s \leq \sum_{s' \in S \setminus \{s\}} s'$; instances of NUMBER PARTITIONING violating this condition may be trivially solved in polynomial time and hence excluding these preserves NP-hardness.

Given S , we construct an instance $(A, G, (u_a)_{a \in A})$ of EF-GC by setting $A = \{a_1, a_2\}$, G is the star with center c and with one leaf l_s for every $s \in S$, and $u_{a_1}((c, l_s)) = u_{a_2}((c, l_s)) = s$ for every $s \in S$. Observe that w.l.o.g. we can assume that every edge is fully assigned

² The reason this does not contradict Observation 1 is that the reduction alters the structure of the input graph.

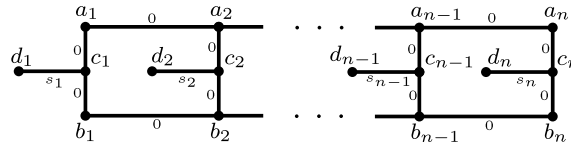


Fig. 3. The graph used in the proof of Theorem 5.

to one of the agents, since otherwise one of the agents would be assigned less than one edge, which would not result in an envy-free assignment. Therefore, every envy-free assignment results in a 2-partition of the edges both having the same total valuation, which shows that S is a Yes-instance of NUMBER PARTITIONING if and only if $(A, G, (u_a)_{a \in A})$ is a Yes-instance of EF-GC. $\square$

Theorem 4. *EF-VDGC is NP-hard even when restricted to instances with two agents with identical utility functions where the graph consists of a matching plus two additional vertices.*

Proof. The reduction is similar to the one employed in Theorem 3 and uses a polynomial-time reduction from the NUMBER PARTITIONING problem. Let S be an instance of NUMBER PARTITIONING, i.e., S is a multi-set of non-negative integers and the aim is to decide whether there is a partition of S into two subsets S_1 and S_2 such that $\sum_{s \in S_1} s = \sum_{s \in S_2} s$. Given S , we construct an instance $(A, G, (u_a)_{a \in A})$ of EF-VDGC as follows. We set $A = \{a_1, a_2\}$ and the graph G has two vertices c_1 and c_2 as well as two vertices l_s and l'_s for every $s \in S$. Moreover, G has all edges between $\{c_1, c_2\}$ and $\{l_s \mid s \in S\}$ and an edge between l_s and l'_s for every $s \in S$. Note that $G - \{c_1, c_2\}$ is a matching. Finally, for every $i \in \{1, 2\}$, we set $u_{a_i}(e) = s$ if $e = (l_s, l'_s)$ for some $s \in S$ and $u_{a_i}(e) = 0$, otherwise. Note that w.l.o.g. we can assume that every edge (l_s, l'_s) is fully assigned to exactly one of the agents since otherwise one agent would only be assigned a part of (l_s, l'_s) (and nothing else). Moreover, by assigning c_i to agent a_i , we can obtain an assignment that assigns any subset of the edges (l_s, l'_s) to agent a_i and all remaining such edges to agent a_{2-i+1} . Since every such assignment is envy-free if and only if the total valuations of the edges assigned to both agents is the same, we obtain that S is a Yes-instance of NUMBER PARTITIONING if and only if $(A, G, (u_a)_{a \in A})$ is a Yes-instance of EF-VDGC. $\square$

Next, we prove that both problems remain hard even when restricted to graphs with constant maximum degree.

Theorem 5. *EF-VDGC and EF-GC are NP-hard even when restricted to instances with two agents with identical valuations where the graph has treewidth 2 and maximum degree 3.*

Proof. Again, we will reduce from NUMBER PARTITIONING. This time, the graph we will construct has maximum degree 3 and it is not hard to verify that has treewidth 2. We will use the same construction to prove both results, this is depicted at Fig. 3. More formally, given an instance of NUMBER PARTITIONING with n positive integers, we create a graph with $4n$ vertices as follows. For every $i \in [n]$, we construct the vertices a_i, b_i, c_i , and d_i and the edges $(a_i, c_i), (c_i, b_i)$ and (c_i, d_i) . In addition, for every $i \in [n-1]$ we create the edges (a_i, a_{i+1}) and (b_i, b_{i+1}) . To complete the construction, we need to specify the valuations of the agents. We create two agents with identical valuations. For every $i \in [n]$, both agents value the edge (c_i, d_i) with s_i . Every other edge has value 0.

To prove the theorem we observe the following. Firstly, note that w.l.o.g. we can assume that every edge (c_i, d_i) is fully assigned to exactly one of the agents since otherwise one agent would only be assigned a part of (c_i, d_i) (and nothing else). Thus, every allocation corresponds to a partition of the n integers.

Finally, we argue that every partition of the n integers, say S_1 and S_2 , can be represented by a corresponding allocation. Towards this, we assign the path $a_1, \dots, a_n$ completely to the first agent and the path $b_1, \dots, b_n$ completely to the second agent. If the i th integer is in S_1 , we assign the edges $a_i c_i$ and $c_i d_i$ completely to the first agent; otherwise we assign the edges $b_i c_i$ and $c_i d_i$ completely to the second agent. The remaining unassigned edges have a valuation of 0 for both agents and can be assigned arbitrarily (while preserving connectivity). This allocation is connected, vertex-disjoint, and—since the sum of numbers in S_1 is precisely the same as the sum of numbers in S_2 —it is also envy-free. Thus, we conclude that there exists an envy-free allocation for the produced instance if and only if there is a solution for the NUMBER PARTITIONING instance. $\square$

Our aim is now to proceed to the aforementioned algorithmic result for EF-VDGC. However, to that end we first need to establish the following technical lemma, which will also be useful for our later considerations. We say that an edge set F is a *splitting set* for an assignment if for each connected component C of $G - F$, there exists an agent α with the following property: every piece of every edge in C belongs to α .

Lemma 6. *Let $I = (A, G, \{u_a\}_{a \in A})$ be an instance of EF-GC (EF-VDGC) and let G be a tree, or a cycle, and let F be a set of edges of G . There is an algorithm that runs in time at most $\mathcal{O}(|A|^{3|F|+1} \cdot |F|^{|A|+1} \cdot Q(|F||A| + |A|))$, where $Q(x)$ denotes the time required to solve a linear program with x variables, and either outputs an envy-free assignment π for which F is a splitting set, or correctly identifies that such an assignment does not exist.*

Proof. Observe that $G - F$ consists of most $|F| + 1$ connected components. Moreover, because F is a splitting set, there are at most $|A|^{|F|+1}$ assignments of these connected components to agents. Note that for every edge e in F there are at most 2 agents that can be assigned a piece of e together with a piece of some other edge. These are the agents that receive the piece $(e, [0, c])$ and the piece $(e, [d, 1])$ for some constant $c, d \in [0, 1]$ with $c \neq 0$ and $d \neq 1$. For each edge e , we guess the agent that gets the piece $(e, [0, c])$ (for some unspecified $c \in (0, 1]$) and say that this is the guess for pair $(e, 0)$; analogously, we guess the agent that gets the piece $(e, [d, 1])$ (for

some unspecified $d \in [0, 1)$ and say that this is the guess for pair $(e, 1)$. This results in $|A|^{2|F|}$ many branches. Consider now one of these assignments π , i.e., π assigns every component of $G - F$ and every endpoint of every edge in F to some agent and let G' be the graph obtained from G after subdividing every edge in F once, i.e., every edge e in F is replaced by two edges e_0 and e_1 corresponding to the pairs $(e, 0)$ and $(e, 1)$. Moreover, for an agent a , let $\pi'(a)$ be the set of edges of G' containing every edge of $G - F$ that is assigned to a plus for every edge $e \in F$ such that $(e, 0)$ or $(e, 1)$ is assigned to a , $\pi'(a)$ contains the corresponding edge e_0 or e_1 in G' .

We now check whether $\pi'(a)$ is connected for every agent a and if not we disregard this branch. Moreover, if $\pi'(a)$ is acyclic, then every edge e of F , whose two endpoints are assigned to a has to be completely assigned to a . Otherwise, G is a cycle and $\pi'(a)$ is equal to G' , which implies that a has to be assigned all but exactly one edge of G entirely and we can guess the edge in F that is not assigned entirely to a .

Let $F' \subseteq F$ be the set of edges not yet fully assigned to any agent. Now, each agent that is not assigned any piece yet, will get a piece *fully inside* some edge of F' . For each such agent we branch on the edge they will be contained in. This results in at most $|F'|^{|A|} \leq |F|^{|A|}$ branches.

At this point there are two possibilities for an agent.

- The agent is assigned to a single edge e of F and its piece is fully inside e .
- The agent is assigned all edges in a subgraph T of G induced by a union of some connected components of $G - F$ and edges of F plus some pieces at the endpoints of some of the edges in F .

Note that if I is an instance of EF-VDGC, then we can check that the trees T_1 and T_2 assigned to two different agents in our branching are vertex disjoint and terminate the branch if it is not the case.

For each branch we now design an instance of Linear Programming (LP) that has a solution if and only if there exists an envy-free assignment consistent with our branching (which can be viewed as a “guess” of the properties of an optimal solution). For each edge $e \in F$ and each agent $a \in A$ such that a is allowed to have a piece of the edge e according to our guess we have a variable x_e^a which represents the length of the piece of the edge e that the agent a gets assigned to. We add the following constraints into the LP:

$$x_e^a \geq 0 \text{ for all } a \in A, e \in F \quad (1)$$

$$\sum_{a \text{ is allowed a piece of } e} x_e^a = 1 \text{ } e \in F. \quad (2)$$

Let $\mathbf{x}$ denote the vector of all variables in some fixed order. Given $\mathbf{x}$, we can compute the value of the piece assigned to an agent a . If agent a is assigned a subtree T and F_T denotes the subset of edges in F such that agent a gets an endpoint of an edge in F . Then for every agent $a' \in A$, we set

$$u_{a'}(a, \mathbf{x}) = u_{a'}(T) + \sum_{e \in F_T} u_{a'}(e) \cdot x_e^a.$$

For an agent a that is fully assigned a piece inside an edge $e \in F$ and every agent a' , we set

$$u_{a'}(a, \mathbf{x}) = u_{a'}(e) \cdot x_e^a.$$

Now for every ordered pair (a, a') of agents $a, a' \in A$, we add the following constraint to the LP:

$$u_a(a, \mathbf{x}) \geq u_{a'}(a', \mathbf{x}). \quad (3)$$

It is rather straightforward to see that if Constraints (1) and (2) are satisfied, then we can assign to each agent a a connected piece of the graph such that the length of the piece the agent a gets inside the edge e is x_e^a and if we guess the subgraph T for the agent a , then a is assigned all edges inside T fully. Moreover, it is easy to see that in this case the utility of the agent a is $u_a(a, \mathbf{x})$. Hence, Constraint (3) implies that such assignment is envy-free. Moreover, assuming an envy-free assignment π_{SOL} exists, one of our branches has to be consistent with π_{SOL} and in that case the algorithm will output a (vertex-disjoint) envy-free assignment. We conclude that if the algorithm does not find a solution in any of the branches then the input is a No-instance.

Overall this procedure requires time in $\mathcal{O}(|A|^{3|F|+1} \cdot |F|^{|A|+1} \cdot Q(|F||A| + |A|))$, where $Q(x)$ denotes the time required to solve a linear program with x variables. The most efficient currently known procedure requires $\mathcal{O}(x^\omega L)$ [48] where ω is the matrix multiplication constant and L is the number of input bits, which in our case is upper-bounded by $|I|$. $\square$

Lemma 6 allows us to solve EF-VDGC on trees by applying initial branching to reach a situation satisfying the preconditions of Lemma 6.

Theorem 7. For each constant $c \in \mathbb{N}$, EF-VDGC restricted to instances with at most c agents is polynomial-time solvable on trees.

Proof. Let π_{SOL} be an arbitrary vertex-disjoint envy-free assignment. Since G is a tree and agents do not share vertices, we claim that there is a set F of at most $|A| - 1$ edges with the following properties: (I) each of the exactly $|F| + 1$ components of $G - F$ is assigned to a single agent, and (II) every edge $e \in F$ is split between the two agents assigned to the components that contain a vertex incident with e plus some of the agents who are not assigned any of these components.

To see the claim, let us root G in some arbitrary vertex r . Let a_r be the unique agent that gets vertex r in some piece of $\pi_{\text{SOL}}(r)$. For every other agent a , there is a unique point x in the closure of pieces of a such that the unique path from x to r does not intersect any piece of a (note that x is not necessarily a vertex and could be a point inside some edge). In other words, there is a unique point x in the closure of pieces of a that is closest to r ; this holds since the union of the pieces assigned to a is connected. Given this point we assign to the agent a a single edge e_a as follows.

- If x is inside some edge f , then we let $e_a = f$.
- If x is a vertex of G and there is a unique edge f that contains a piece of a , then we let $e_a = f$.
- Else x is a vertex of G incident to at least 2 edges that get a piece of a . In this case, we let e_a be the unique edge incident on x on the unique $x - r$ path in G .

Let $F = \{e_a \mid a \in A \setminus \{a_r\}\}$. We show that each connected component of $G - F$ is assigned to a single agent. The rest of the claim then follows straightforwardly from the connectivity of the pieces assigned to single agents. Let T be a subtree of G induced by some connected component of $G - F$. We can root T in the unique vertex that is “closest” to r . That is the vertex r_T such that the unique $r_T - r$ path does not contain any vertex of T . We show that all edges of T are assigned to some agent a such that e_a is the unique edge from r_T to its parent in G . We distinguish two possibilities. If all agents contain a piece in at most one edge incident with r_T , then it is easy to see that all edges incident with r_T are in F . Therefore, T is a singleton and the claim follows. Else, if there is an agent a that gets pieces in at least two edges incident on r_T , then e_a is the edge from r_T to its parent and a is the unique such agent; i.e., all other agents get pieces in at most one edge incident on r_T .

We show by induction on the distance from r_T that all edges in T are fully assigned to a by π_{SOL} . For the base case, we note that the edges incident to r_T in T are assigned to a by π_{SOL} . This holds because if any other agent $a' \neq a$ were to contain a piece of an edge f incident to r_T , then $f \in F$ and in particular $f \notin E(T)$.

To complete the induction, let us consider a vertex $v \in V(T)$ at distance i from r_T . We know, by the induction hypothesis, that the unique edge f between v and its parent in T is fully assigned to a . If there is an agent $a' \neq a$ that contains pieces in at least 2 edges incident to v , then $e_{a'} = f$, which is not possible as $f \in E(T)$. Therefore, every agent $a' \neq a$ contains a piece in at most one edge incident with v . Now, if a' contains a piece inside edge f' incident on v , then $e_{a'} = f'$ and this would contradict $f' \in E(T)$. Hence all edges incident with v in T are also fully assigned to a and the claim follows by induction. We note that the same discussion works even if $r_T = r$, in which case we show that T is assigned to a_r analogously.

We can now enumerate all of the at most $|E(G)|^{|A|} \leq |V(G)|^{|A|}$ possible sets of edges $F \subseteq E(G)$ such that $|F| \leq |A| - 1$. The theorem then follows by applying Lemma 6 to G and F .

Using the notation and the running time bound from the proof of Lemma 6, it follows that the overall complexity lies in $\mathcal{O}(|V(G)|^{|A|} \cdot |A|^{3|A|-2} \cdot (|A| - 1)^{|A|+1} \cdot Q(|A|^2))$, where $Q(x)$ denotes the time required to solve a linear program with x variables. $\square$

To complete our understanding of EF-GC and EF-VDGC for instances with only boundedly-many agents, we show that both problems are also polynomial-time solvable on graphs of maximum degree 2 (i.e., cycles) and that the former is polynomial-time solvable on bounded-degree trees. These results follow a similar approach as the proof of Theorem 7, which is a combination of initial branching and Lemma 6.

Theorem 8. *For each pair of constants $c, d \in \mathbb{N}$, EF-GC restricted to instances with at most c agents is polynomial-time tractable on trees with maximum degree at most d .*

Proof. We show that there are at most $|A|$ vertices and edges of G that can be in a piece of more than one agent. Given this claim, we can enumerate all of the at most $(2 \cdot |V(G)| - 1)^{|A|}$ possible sets of vertices $U \subseteq V(G)$ and edges $F \subseteq E(G)$ such that $|U \cup F| \leq |A|$. Let F' be the union of F and the edges incident with vertices in U . Note that $|F'| \leq |A| \cdot d$, where d is the maximum degree of G . The theorem then follows by applying Lemma 6 to G and F' in each branch.

The rest of the proof is devoted to showing that there are at most $|A|$ many vertices and edges of G than can be in a piece of more than one agent. Let π_{SOL} be an arbitrary envy-free assignment, and let U and F be the set of vertices and edges, respectively, which lie in a piece of more than one agent w.r.t. π_{SOL} . That is, for every vertex $u \in U$ there exist two different agents a_1 and a_2 such that both $\pi_{\text{SOL}}(a_1)$ and $\pi_{\text{SOL}}(a_2)$ contain a piece that contains u . Similarly, for every edge $e \in F$ there are two agents a_1 and a_2 such that both $\pi_{\text{SOL}}(a_1)$ and $\pi_{\text{SOL}}(a_2)$ contain a piece of the edge e . We will construct an injective function $f : U \cup F \rightarrow A$, that is, we show that we can assign to every element (vertex or edge) in $U \cup F$ a unique agent.

Let us root G in some arbitrary vertex r . For the rest of the proof, for a vertex v , we will denote by p_v the parent of the vertex v in the rooted tree G rooted in r . For a vertex $u \in U$, at most one agent contains some piece of the edge up_u from u to its parent p_u that contains the vertex u together with some non-empty part of the edge up_u . Since there are at least two agents whose pieces contain the vertex u , there is at least one other agent a such that a does not get a piece of the edge up_u , but gets a piece of some other edge that contains the vertex u ; note that the same applies if u is equal to the root r of G . We let $f(u) = a$. For an edge $e \in F$ let $e = up_u$, where p_u is the parent of u . We let $f(e) = a$, where a is the unique agent that gets the piece of e containing u together with some non-empty part of e . Note that since at least two agents contain a piece of the edge e , it follows that the piece of agent a does not contain p_u .

Now let x and y be two different elements of $U \cup F$. We show that $f(x) \neq f(y)$. Clearly, if $f(x) = f(y) = a$, then the piece assigned to the agent a under π_{SOL} contains a piece of both x and y . We will show that if an agent a such that $\pi_{\text{SOL}}(a)$ contains piece of both x and y , then either $f(x) \neq a$ or $f(y) \neq a$. Let u denote the vertex x if $x \in U$, otherwise let u be the vertex such that x is the edge up_u in F . Similarly, let v be y if $y \in U$, else let v be the vertex such that y is the edge vp_v . Note that if $f(x) = a$, then $\pi_{\text{SOL}}(a)$ contains a piece containing the vertex u . Similarly, if $f(y) = a$, then $\pi_{\text{SOL}}(a)$ contains v . Since the piece of the agent a is connected, it follows that $\pi_{\text{SOL}}(a)$ contains a path from u to v , moreover since G is a tree, this path is unique. Now note that if v is an ancestor of u , then the unique path from u to v contains the edge up_u and from the definition of f it follows $f(x) \neq a$. If u is an ancestor of v , then the unique path from u to v in G contains the edge vp_v , and $f(y) \neq a$. Finally, if v is not ancestor of u and u is not an ancestor of v , then the unique path from u to v contains both edges up_u and vp_v , hence $f(x) \neq a$ and $f(y) \neq a$.

Using the notation and the running time bound from the proof of Lemma 6, it follows that the overall complexity lies in $\mathcal{O}((2|V(G)| - 1)^{|A|} \cdot |A|^{3|A|d+1} \cdot (|A|d)^{|A|+1} \cdot Q(|A|^2d + |A|))$, where $Q(x)$ denotes the time required to solve a linear program with x variables. $\square$

Theorem 9. *For each constant $c \in \mathbb{N}$, EF-GC and EF-VDGC restricted to instances with at most c agents is polynomial-time solvable on cycles.*

Proof. It is straightforward to see that in any partition of a cycle graph G into $|A|$ connected pieces there are at most $|A|$ edges that are assigned to two different agents. The theorem then follows by branching on all $|V(G)|^{|A|}$ many possibilities of at most $|A|$ many edges shared by more than one agent. The theorem then follows by applying Lemma 6 in each branch.

Using the notation and the running time bound from the proof of Lemma 6, it follows that the overall complexity lies in $\mathcal{O}(|V(G)|^{|A|} \cdot |A|^{3|A|+1} \cdot |A|^{3|A|+1} \cdot Q(|A|^2 + |A|))$. $\square$

4. Instances with few edges

In the section we provide an algorithm showing that both EF-GC and EF-VDGC can be solved in polynomial time if the graph contains a constant number of edges; put differently, both problems belong in XP parametrized by the number of edges in the input graph G .

Theorem 10. *EF-GC and EF-VDGC can be solved in time $|A|^{\mathcal{O}(|E(G)|^2)}$.*

The algorithm can be divided into three main steps, which we first describe below in order to gain some intuition. Then, we provide a formal explanation of the algorithm and we prove its correctness.

For the remainder of this section, we fix an instance I of EF-GC or EF-VDGC given by the set of agents A , graph $G = (V, E)$, and utilities $u_a : E \rightarrow \mathbb{Q}^+$ for each agent $a \in A$. Denote $k = |E(G)|$.

Guessing the assignment of significant agents We start with a direct brute-force branching over all “significant” agents that essentially define a solution, if it exists. There are four types of significant agents, where each type defines specific constraints.

Guessing procedure

1. For every endpoint of an edge, guess which agent gets it; denote A_V the set of agents that get an endpoint.
2. For every edge e , guess how many agents n_e will get a piece fully contained in e .
3. For every pair of edges (e, f) guess the *edge envy-critical* agent $a_{(e,f)} \notin A_V$ that gets a piece fully contained in e (hence we allocate them a piece in e) that has the largest envy towards pieces fully contained in f .
4. For every edge e and every agent $a \in A_V$, guess the *vertex envy-critical* agent $a' = a'_{e,a} \notin A_V$ that gets a piece fully contained in e (hence we allocate them a piece in e) and has the largest envy towards a .

While it is not too hard to bound the guesses for Steps 1–2 by $|A|^{\mathcal{O}(k)}$, and the guesses of edge envy-critical agents of Steps 3 by $|A|^{k^2}$ (we provide detailed explanations of these later), bounding the number of vertex envy-critical agents of Steps 4 is far from obvious; a highly non-trivial part of Steps 4 is to actually impose some ordering on agents outside of A_V in order to be able to define vertex envy-critical agents and we use Proposition 1 in order to achieve this. As we prove in Lemma 11, in Steps 4 again $|A|^{\mathcal{O}(k)}$ guesses suffice. Hence, overall, we need at most $|A|^{\mathcal{O}(k^2)}$ guesses. Note that not all guesses are reasonable, e.g., we should not allow the same agent to be edge envy-critical for two different edges. We perform sanity checks and we discard all the “unreasonable” guesses; we explain in detail how to do this. Each branch of our guesses already provides useful information about a hypothetical sought-after solution—but not enough to solve the problem. Crucially, though, as we prove, every solution to the original instance corresponds to one of the branches.

Cutting the pieces suitably Our aim in the second step will be to construct, for each branch, a Linear Program (LP) to determine the exact lengths of all the pieces in an envy-free partitioning. In particular, if the branch, i.e., our guess, corresponds to a solution, then we require that the LP outputs a partitioning that can be matched to agents in a way that also produces a solution.

Edge-cutting LP

Given a branch of the guessing procedure, construct the edge-cutting LP where:

- every edge is fully allocated (Constraints of type (4) and (5));
- agents in A_V do not envy each other (Constraints of type (6));
- agents in A_V do not envy agents not in A_V (Constraints of type (7));
- agents not in A_V do not envy each other (Constraints of type (8)) — for this constraint we use the guessed edge envy-critical agents;
- agents not in A_V do not envy agents in A_V (Constraints of type (9)) — for this constraint we use the guessed vertex envy-critical agents.

Assigning the pieces Finally, if the constructed LP has a solution, we need to decide whether we can allocate the constructed pieces and get a solution; if the LP is infeasible, or we *cannot* allocate the pieces in an envy free way, then we discard the guessing and we proceed to the next one. So, based on the guesses and a solution to the constructed LP, we do the following:

- we first assign the correct pieces to the agents that we have already made a guess for;
- we design an instance of bipartite matching that matches the remaining unassigned agents with the pieces given by the LP instance. If a matching exists, we are guaranteed to have found a solution; if not, then our branch does not correspond to a valid solution.

Matching Procedure

1. If the constructed LP has a feasible solution, create an auxiliary bipartite graph $H = (X \uplus Y, F)$ where:
 - every vertex in X corresponds to a piece of an edge that has not been allocated to any agent yet;
 - every vertex in Y corresponds to an agent that has not been allocated a piece yet;
 - there is an edge $(x, y) \in F$, if among all pieces, agent y has maximum value for piece x .
2. If graph H admits a perfect matching, then return the solution; otherwise discard the guess and proceed to the next one.

Putting everything together We are ready to describe our algorithm in full by essentially combining the three procedures we have described above.

Algorithm for Graph Cutting with Few Edges

- For every possible guess of the Guessing procedure do:
 - Construct the Edge-cutting LP.
 - If the LP has a solution, use the Matching Procedure to compute an assignment of the pieces to the agents, if it exists.

Next we provide the formal explanation of the algorithm and we prove its correctness. In order to explain better how to choose the edge and vertex envy-critical agents, we split the guessing into two steps, and we define them in [Section 4.2](#).

4.1. Initial guessing of A_V and n_e

We can now start with the branching phase. Let us assume $\mathcal{I}$ admits an envy-free assignment π_{SOL} into some partition of G into pieces; we will describe the branching as a series of “guesses” of the properties of this solution $\mathcal{I}$ and its interactions with G .

First, observe that for each edge e there are at most 2 agents that can be assigned a piece of the edge e together with a piece of some other edge. These are the agents in π_{SOL} that receive the piece $(e, [0, c])$ and the piece $(e, [d, 1])$ for some constant $c, d \in [0, 1]$. For each edge e , we guess the agent that gets the piece $(e, [0, c])$ (for some unspecified $c \in [0, 1]$) and say that this is the guess for pair $(e, 0)$; analogously, we guess the agent that gets the piece $(e, [d, 1])$ (for some unspecified $d \in [0, 1]$) and say that this is the guess for pair $(e, 1)$. This results in $|A|^{2k}$ many branches. Let A_V be the set of the at most $2k$ agents guessed in the previous step.

All the remaining agents are assigned by π_{SOL} a single piece (of some edge e). For every such agent, we say that it gets a piece *fully contained inside edge e* . While it is too computationally expensive to guess precisely which agents get a piece fully contained in an edge e , we will guess the number of agents that get a piece fully contained in an edge e . This results in $|A| + 1$ many guesses for each edge, amounting to a branching factor of at most $(|A| + 1)^k$. Let us denote by n_e the number of agents that get a piece fully contained in the edge e . We now perform a set of sanity checks on our branching; in particular, we discard branches which do not fulfill the following conditions:

1. $|A_V| + \sum_{e \in E(G)} n_e = |A|$, i.e., all agents get a piece.
2. For every agent $a \in A_V$, the guesses of pieces assigned to a form a connected subset of G . More formally, whenever our branch assigns an agent a to two distinct pairs (e, i) and (f, j) , where $e, f \in E(G)$ and $i, j \in \{0, 1\}$, there must exist a path $P = e_1 e_2 \dots e_q$ from $e[i]$ to $f[j]$ such that for each $i \in [q]$ it holds that (I) $n_{e_i} = 0$ and (II) the agent a is also the guess for both $(e_i, 0)$ and $(e_i, 1)$.
3. In the case of EF-VDGC, we will also verify that the branching corresponds to a vertex-disjoint partition. In particular, for each vertex v let $E_{v,0}$ be the set of edges incident to v and a vertex preceding v in the ordering and $E_{v,1}$ be the set of edges incident to v and a vertex succeeding v in the ordering. We check that there is a single agent a such that for each edge $e \in E_{v,0}$, a is the guess for $(e, 0)$, and at the same time for each edge $e' \in E_{v,1}$, a is the guess for $(e', 1)$.

4.2. Additional guesses and cutting the pieces via linear programming

We can now begin describing the instance of LP that we create for each branch and we will use to determine the partition. For this, it will be useful to observe that agents fully contained in the same edge must receive a segment of the same length.

Observation 2. Let $e \in E(G)$ be an edge and $a_1, a_2 \in A$ be two agents fully contained inside edge e such that $\pi_{\text{SOL}}(a_1) = \{(e, [x_1, y_1])\}$ and $\pi_{\text{SOL}}(a_2) = \{(e, [x_2, y_2])\}$. Then, $|y_1 - x_1| = |y_2 - x_2|$.

Proof. Since $u_{a_1}(\pi_{\text{SOL}}(a_1)) = |y_1 - x_1| \cdot u_{a_1}(e)$ and $u_{a_1}(\pi_{\text{SOL}}(a_2)) = |y_2 - x_2| \cdot u_{a_1}(e)$, envy-freeness implies $|y_1 - x_1| \geq |y_2 - x_2|$. Similarly, $u_{a_2}(\pi_{\text{SOL}}(a_1)) = |y_1 - x_1| \cdot u_{a_2}(e)$ and $u_{a_2}(\pi_{\text{SOL}}(a_2)) = |y_2 - x_2| \cdot u_{a_2}(e)$, hence envy-freeness implies $|y_1 - x_1| \leq |y_2 - x_2|$. $\square$

For each edge $e \in E(G)$, the LP instance will have variables x_e^0, δ_e , and x_e^1 . The variable δ_e represents the length of each piece $(e, [c, d])$ assigned to any agent that gets a piece fully inside e . The variable x_e^i represents the length of the piece of the edge e that was assigned to the agent $a \in A_V$ which is the guess for pair (e, i) . We start by adding the following two constraints saying that all pieces have non-negative length and the sum of lengths of pieces on each edge is exactly 1.

$$x_e^0, \delta_e, x_e^1 \geq 0 \quad (4)$$

$$x_e^0 + n_e \cdot \delta_e + x_e^1 = 1. \quad (5)$$

For each agent $a \in A_V$, let Pieces_a denote the set of pieces that a is the guess for, formally:

$$\text{Pieces}_a = \{(e, i) \mid e \in E(G), i \in \{0, 1\}, \text{ and } a \text{ is the guess for } (e, i)\}.$$

Given the intended meaning of variables x_e^0, δ_e , and x_e^1 , we can now add constraints to guarantee envy-freeness between agents in A_V . For all $a, a' \in A_V$ we create the constraint

$$\sum_{(e,i) \in \text{Pieces}_a} u_a(e) \cdot x_e^i \geq \sum_{(e',i') \in \text{Pieces}_{a'}} u_a(e') \cdot x_{e'}^{i'}, \quad (6)$$

and for every $a \in A_V$ and $e \in E(G)$ we create the constraint

$$\sum_{(f,i) \in \text{Pieces}_a} u_a(f) \cdot x_f^i \geq u_a(e) \cdot \delta_e. \quad (7)$$

Next, we describe how to guess the edge envy-critical agents. For every (ordered) pair of edges e, f such that $n_e > 0$ and $n_f > 0$, we need to guarantee that agents that get a piece of length δ_e do not envy agents with pieces fully inside edge f . If we knew that $a \in A \setminus A_V$ gets a piece fully inside e , then for this agent we could express this via the constraint $u_a(e) \cdot \delta_e \geq u_a(f) \cdot \delta_f$. Unfortunately, we do not know which agents get a piece fully inside e and cannot obtain this information by exhaustive branching in view of our time bounds.

To overcome this obstacle, let us order the agents in $A \setminus A_V$ by the ratio $\frac{u_a(e)}{u_a(f)}$ and consider two agents a_1, a_2 such that

$$\frac{u_{a_1}(e)}{u_{a_1}(f)} \geq \frac{u_{a_2}(e)}{u_{a_2}(f)}.$$

It is easy to see that $u_{a_2}(e) \cdot \delta_e \geq u_{a_2}(f) \cdot \delta_f$ implies $u_{a_1}(e) \cdot \delta_e \geq u_{a_1}(f) \cdot \delta_f$. Hence to capture the desired constraint it will be sufficient to guess, for each ordered pair of edges (e, f) the agent $a_{(e,f)}$ that is assigned a piece fully inside e (by π_{SOL}), and has the smallest value for the fraction $\frac{u_{a_{(e,f)}}(e)}{u_{a_{(e,f)}}(f)}$ among all the agents that are assigned a piece fully inside e . Intuitively, this corresponds to guessing an edge envy-critical agent a : among all the agents fully contained inside edge e , the agent a is “closest” to envying agents fully contained inside edge f . Note that the guessed edge envy-critical agents will later preclude some agents from receiving a piece of e (in particular, those that precede a in the linear order defined by the fractions).

The procedure described above introduces at most k^2 many guesses of agents, which amounts to an additional branching factor of at most $|A \setminus A_V|^{k^2}$. For each agent $a_{(e,f)}$, we then add the following constraint to the LP instance

$$u_{a_{(e,f)}}(e) \cdot \delta_e \geq u_{a_{(e,f)}}(f) \cdot \delta_f. \quad (8)$$

We also perform additional consistency checks for this branching. First of all, we discard branches which select the same agent as being edge envy-critical in multiple pieces (i.e., if $e \neq e'$ then we require $a_{(e,f)} \neq a_{(e',f')}$, regardless of f and f'). Moreover, since we have guessed at most $k - 1$ agents for an edge e , we also check that the intended meaning of the choice of the agent $a_{(e,f)}$ is satisfied so far: for all triples of edges e, f, f' we check that

$$\frac{u_{a_{(e,f')}}(e)}{u_{a_{(e,f')}}(f)} \geq \frac{u_{a_{(e,f)}}(e)}{u_{a_{(e,f)}}(f)}. \quad (A)$$

In what follows we explain how to guess vertex envy-critical agents and how we can bound the number of necessary guesses. At this point we have added constraints which prevent—assuming our guesses were correct—an agent in A_V from envying any other agent, and agents outside of A_V from envying each other. Finally, for every edge e and every agent $a \in A_V$ we would like to guarantee that the agents that get a piece fully inside e do not envy the agent a . Similarly as before, for each specific agent a' that gets a piece fully inside e we could hypothetically ensure this via the constraint

$$u_{a'}(e) \cdot \delta_e \geq \sum_{(f,i) \in \text{Pieces}_a} u_{a'}(f) \cdot x_f^i.$$

However, we again do not know the agents that are assigned a piece fully inside e . Unfortunately, while for two edges e and f it was not too difficult to define and identify envy-critical agents and write linear constraints only for those, when comparing the envy of agents fully contained inside edge e towards an agent $a \in A_V$ that receives multiple pieces of edges, the notion of “envy-criticality” we need depends on the size of the pieces a gets from each edge. In particular, there is no fixed total ordering of the agents that allows us to define envy-criticality. To give a concrete example of this issue, for two different instantiations of the x_f^i variables, say $x_f^i := c_f^i$ and $x_f^i := d_f^i$, and two agents a_1 and a_2 it may hold that

$$\frac{\sum_{(f,i) \in \text{Pieces}_{a_1}} u_{a_1}(f) \cdot c_f^i}{u_{a_1}(e)} \geq \frac{\sum_{(f,i) \in \text{Pieces}_{a_2}} u_{a_2}(f) \cdot c_f^i}{u_{a_2}(e)},$$

but

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot d_f^i}{u_{a_1}(e)} \leq \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot d_f^i}{u_{a_2}(e)}.$$

On the other hand, the assignment π_{SOL} does define some specific instantiation of x_e^i 's for which there is a (not necessarily strict) total ordering on the agents a' in A capturing how “close” they are to envying a , i.e., based on the value of

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a'}(f) \cdot x_f^i}{u_{a'}(e)}.$$

While we have no way of computing which total ordering arises from the hypothetical assignment π_{SOL} , we will later (in [Lemma 11](#)) use [Proposition 1](#) to show that only $|A|^{\mathcal{O}(k)}$ many such orderings are possible, and moreover that we can enumerate all of these in time $|A|^{\mathcal{O}(k)}$. For now, let us complete the description of the LP with this in mind. Since the number of relevant orderings is bounded by $|A|^{\mathcal{O}(k)}$, we can apply branching to guess the ordering that arises from a hypothetical targeted assignment. At that point we can also guess, for each edge e and agent $a \in A_V$, the *envy-critical* (according to this ordering) agent $a_{e,a}$ that is assigned to the edge e and envies the agent a the most—and later use this guess to preclude some agents from being fully contained inside edge e .

For each guess, we add constraints to the LP which will ensure that the guess will be consistent with whatever solution the LP produces. In particular, we add the constraint

$$u_{a'}(e) \cdot \delta_e \geq \sum_{(f,i) \in \text{Pieces}_a} u_{a'}(f) \cdot x_f^i \quad (9)$$

for every agent a' that envies a at most as much as $a_{e,a}$ according to the guessed ordering. More precisely, with each guess we will get some instantiation $x_e^i = y_e^i$ witnessing this guess, and we will insert a copy of Constraint (9) for every agent a' satisfying the following property:

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_{e,a}}(f) \cdot y_f^i}{u_{a_{e,a}}(e)} \geq \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a'}(f) \cdot y_f^i}{u_{a'}(e)}. \quad (B)$$

Our last task in this step is to provide a way to perform the branching over total orders described above. Recall that each instantiation of the variables x_e^i , for $i \in \{0, 1\}$ and $e \in E(G)$, gives rise to a set of total orderings of all agents in A . In particular, each ordering is associated with precisely one pair $(a \in A_V, e \in E(G))$. Here, each ordering captures the relative envy towards a under the assumption that the agents would be assigned to e (see Inequality (B)). We call a set of such orderings a *portfolio*. Moreover, since π_{SOL} also corresponds to an instantiation of the variables x_e^i , it too gives rise to a set of total orderings, which we call a *portfolio* of π_{SOL} .

Lemma 11. *It is possible to construct, in time $A^{\mathcal{O}(k)}$, a set $\mathcal{R}$ of at most $k^k \cdot A^{\mathcal{O}(k)}$ -many portfolios which is guaranteed to contain the portfolio of π_{SOL} .*

Proof. Let us define the set $\mathcal{P}$ of $k \cdot |A|^3$ many linear polynomials in the variables x_e^i for every $e \in E(G)$ and $i \in \{0, 1\}$. Namely, for each edge $e \in E(G)$, agent $a \in A_V$, and pair of agents $a_1, a_2 \in A$, let

$$P_{(a_1, a_2)}^{(e, a)} = \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot x_f^i}{u_{a_1}(e)} - \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot x_f^i}{u_{a_2}(e)}.$$

Note that

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot x_f^i}{u_{a_1}(e)} > \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot x_f^i}{u_{a_2}(e)}$$

if and only if $\text{sign}(P_{a_1, a_2}^{(e, a)}(\vec{x})) = 1$,

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot x_f^i}{u_{a_1}(e)} = \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot x_f^i}{u_{a_2}(e)}$$

if and only if $\text{sign}(P_{a_1, a_2}^{(e, a)}(\vec{x})) = 0$, and $\text{sign}(P_{a_1, a_2}^{(e, a)}(\vec{x})) = -1$ otherwise. The assignment π_{SOL} gives concrete values for x_e^i 's and hence defines some point w in $\mathbb{R}^{2k}$. Depending on the values of the polynomials $P_{a_1, a_2}^{(e, a)}$ in w , the point w defines a sign vector σ . The set of all points that define the same sign vector σ is a cell of $\mathcal{P}$ over $\mathbb{R}^{2k}$; observe that each sign vector completely describes one possible ordering of the agents w.r.t. these inequalities (or, equivalently, Inequality (B)).

[Proposition 1](#) provides an algorithm that computes a point in each cell of $\mathcal{P}$ over $\mathbb{R}^{2k}$ in time $k^k \cdot |A|^{\mathcal{O}(k)}$, which immediately gives the same upper bound on the number of cells. Since w is a point in some cell, one of the points identified by the algorithm is in the same cell as w and defines the same sign vector σ . $\square$

To formalize the description provided earlier, we now branch over all at most $k^k \cdot |A|^{\mathcal{O}(k)}$ many portfolios obtained from [Lemma 11](#). Given some point $y \in \mathbb{R}^{2k}$, we guess for each pair of edge $e \in E(G)$ and agent $a \in A_V$ an agent $a_{e,a}$ as described above and introduce the LP instance constraints described in (9). Finally, similarly as after introducing Constraints (8), we can again check that the intended meaning of guessed agents for each edge e hold by checking Inequalities (A) and (B) for every pair of agents guessed for each edge e . If at least one of the inequalities does not hold, then we reject the branch. This finishes the construction of the LP instance.

4.3. Allocating the pieces via bipartite matching

Now, we can solve each LP instance in at most cubic time [49]. If the instance is unsatisfiable, then the algorithm rejects this guess and continues to the next one. Else, given an LP solution x , we can assign the values for the agents that we already guessed, i.e., every agent in A_V gets the piece that contains the corresponding vertex (or vertices), and edge envy-critical and vertex envy-critical agents get a piece within the corresponding guessed edge. So, for each agent $a \in A_V$, we let

$$\pi(a) = \bigcup_{(e,0) \in \text{Pieces}_a} \{(e, [0, x_e^0])\} \cup \bigcup_{(e,1) \in \text{Pieces}_a} \{(e, [1 - x_e^1, 1])\}.$$

Every other agent a that we identified via a guess was fully contained inside some particular edge e . Moreover, by Constraint (5), we can split the interval $[x_e^0, 1 - x_e^1]$ into n_e pieces of length δ_e ; note that if $n_e > 0$ then δ_e cannot be equal to 0. Let $I_a \subseteq [x_e^0, x_e^1]$ be any of the pieces that has not been assigned to another agent yet and let $\pi(a) = (e, I_a)$.

Finally, we are left with some unassigned agents and some unassigned pieces of the graph, each consisting of a single piece of an edge. Since, $|A_V| + \sum_{e \in E(G)} n_e = |A|$, the number of unassigned pieces equals the number of unassigned agents. Moreover, since at this point we have a concrete partition, for every pair of agent a and piece (e, I) we can in polynomial time check whether a would envy a piece in the partition or not (since this check can be performed without knowing the assignments of the other agents); in the former case we say that a is *compatible* with (e, I) , and otherwise we say that they are *incompatible*.

Thus, we create an auxiliary bipartite graph $H = (X \uplus Y, F)$ such that each vertex in X is identified with an unassigned agent, each vertex in Y is identified with an unassigned piece, and there is an edge between an agent $a \in X$ and a piece $(e, I) \in Y$ if and only if they are compatible. We compute a maximum matching M in H using, e.g., the Hopcroft-Karp algorithm in time at most $\mathcal{O}(|A|^3)$. If M is not a perfect matching, then we reject the branch of our algorithm and try another branch. Else for each unassigned agent a , we let $\pi(a) = M(a)$, where $M(a)$ denotes the piece $(e, I) \in Y$ matched with the agent $a \in X$ by the matching M . In this case the algorithm outputs Yes and optionally also the assignment π as a witness. If none of the branches lead to a positive outcome, the algorithm outputs No.

This concludes the description of the algorithm, and we now proceed to establishing its correctness.

Lemma 12. *If the above algorithm outputs an assignment π , then the assignment π is envy-free.*

Proof. For the ease of presentation, we assume that for $i \in \{0, 1\}$ and $e \in E(G)$, x_e^i and δ_e are the values of the same-name variables that are output of the LP solver in the branch that leads to the assignment π . By the construction of the assignment π we have that for every agent $a \in A_V$

$$u_a(\pi(a)) = \sum_{(e,i) \in \text{Pieces}_a} u_a(e) \cdot x_e^i,$$

and every other agent $a \in A \setminus A_V$ gets a piece that is fully inside some edge e and has length δ_e , hence

$$u_a(\pi_a) = u_a(e) \cdot \delta_e.$$

The fact that for every (e, i) , $e \in E(G)$ and $i \in \{0, 1\}$, we guessed an agent in A_V , that $|A_V| + \sum_{e \in E(G)} n_e = |A|$, and that all constraints of type (5) are satisfied implies that $\pi(A)$ is indeed a partition of G . Constraints of type (6) and (7) guarantee that the agents in A_V do not envy any other agent. Moreover, because for every edge and a pair of agents assigned to that edge in our guessing part we check that Inequalities (A) and (B) hold, and because Constraints (8) and (9) are satisfied, we get that all agents (partially) assigned in our branching phase do not envy any other agent. Finally, all remaining agents are assigned by the matching algorithm, which pairs them only with pieces they value the most and hence none of the agents assigned by the matching algorithm envies any other agent. $\square$

Lemma 13. *If I admits an envy-free assignment π_{SOL} , then the above algorithm will output an envy-free assignment.*

Proof. We will first describe the branch of our algorithm that corresponds to the assignment π_{SOL} , explain why we do not reject this branch and show that the LP we obtain in this branch has a solution. Afterwards, we show that any solution to this LP leads to an envy-free assignment.

Given π_{SOL} , we can find the set of agents A_V that get the endpoint pieces of each edge, as well as for each agent $a \in A_V$ we know precisely which endpoint the agents gets. For agent $a \in A_V$ we denote by Pieces_a , in the same way as in the description of the algorithm, the set of pairs (e, i) such that if $i = 0$, then $\pi_{\text{SOL}}(a)$ contains piece $(e, [0, c])$, for some $c \in [0, 1]$, and if $i = 1$, then $\pi_{\text{SOL}}(a)$ contains piece $(e, [d, 1])$, for some $d \in [0, 1]$. Note that in one of the branches of our algorithm, we get precisely same set of agents A_V with precisely the same Pieces_a for each agent $a \in A_V$. Moreover, each of these agents induces a connected subset of G , hence we did not reject the branch for connectivity reasons. Given this set of pieces, we can already assign the values for x_e^i 's in the LP instance. If $(e, 0) \in \text{Pieces}_a$ and $(e, [0, c])$ is a piece in $\pi_{\text{SOL}}(a)$ such that $c \neq 1$, then we let $x_e^0 = c$, otherwise if $c = 1$, then we let $x_e^0 = \frac{1}{2}$. Similarly, if $(e, 1) \in \text{Pieces}_a$ and $(e, [c, 1])$ is a piece in $\pi_{\text{SOL}}(a)$ such that $c \neq 0$, then we let $x_e^1 = 1 - c$, otherwise if $c = 0$, then we let $x_e^1 = \frac{1}{2}$. Note that

$$u_a(\pi_{\text{SOL}}(a)) = \sum_{(e,i) \in \text{Pieces}_a} u_a(e) \cdot x_e^i.$$

Now, for each edge $e \in E(G)$, we let n_e be the number of agents that, according to π_{SOL} , get a piece fully inside the edge e that does not contain any endpoint of e . By [Observation 2](#), all such pieces have the same length and we let δ_e denote the length of the interval in such piece. For technical reasons, if $n_e = 0$, we let $\delta_e = 0$. For any agent a that gets a piece fully inside the edge e we get

$$u_a(\pi_{\text{SOL}}(a)) = \delta_e \cdot u_a(e).$$

Observe that in one of our branches we guessed precisely same numbers of agents per edge and $|A_V| + \sum_{e \in E(G)} n_e = |A|$, so we did not reject this branch. Moreover, since π_{SOL} is an envy-free assignment and every agent values the whole graph 1, then each agent gets a non-trivial piece. Hence, the given assignment of x_e^i 's and δ_e 's satisfy all constraints of type (4). Since $\pi_{\text{SOL}}(A)$ is a partition of G , we get also that Constraint (5) is satisfied for every edge e . Since $\pi_{\text{SOL}}(A)$ is envy-free, we get that all constraints of types (6) and (7) are satisfied.

Now for an ordered pair of edges (e, f) , if $n_e \geq 1$, then there is an agent $a_{(e,f)}$ that minimizes the value of the fraction $\frac{u_{a_{(e,f)}}(e)}{u_{a_{(e,f)}}(f)}$ among all the agents assigned a piece fully inside e . Note that this agent might not be unique as several agents might have the same value of the fraction, however in one of the branches we guessed the agents $a_{(e,f)}$ among the ones that minimizes the fraction. Since π_{SOL} is envy-free, all constraints of type (8) are satisfied. Moreover, in a correct branch we also satisfy all inequalities of type (A), so we do not reject this branch because of some inequality of type (A).

Finally, the x_e^i 's define a point in $\mathbb{R}^{2k}$ that belong to some cell of $\mathcal{P} = \bigcup_{e \in E(G), a \in A_V, a_1, a_2 \in A} \{P_{a_1, a_2}^{(e, a)}\}$ over $\mathbb{R}^{2k}$. The algorithm branches over all cells of $\mathcal{P}$ that contain a point and hence we branch on some point defined by y_e^i 's from the same cell as x_e^i . It follows that for each agent $a \in A_V$, every edge $e \in E(G)$ and every pair of agents a_1, a_2 we get

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot x_f^i}{u_{a_1}(e)} \geq \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot x_f^i}{u_{a_2}(e)}$$

if and only if

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_1}(f) \cdot y_f^i}{u_{a_1}(e)} \geq \frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_2}(f) \cdot y_f^i}{u_{a_2}(e)}.$$

Now, for each edge $e \in E(G)$ such that $n_e \geq 1$ and each agent $a \in A_V$, there is an agent $a_{e,a}$ that maximizes the fraction

$$\frac{\sum_{(f,i) \in \text{Pieces}_a} u_{a_{e,a}}(f) \cdot x_f^i}{u_{a_{e,a}}(e)}$$

among all the agents that are assigned a piece fully inside e . Since agent $a_{e,a}$ is assigned a piece fully inside e by π_{SOL} , we get that

$$u_{a_{e,a}}(e) \cdot \delta_e \geq \sum_{(f,i) \in \text{Pieces}_a} u_{a_{e,a}}(f) \cdot x_f^i.$$

It follows that in the branch where we guessed $a_{e,a}$ correctly all agents that satisfy Inequality (B) also satisfy Constraint (9). Moreover, in the correct branch, we do not reject the branch because of inequalities of types (A) and (B). It follows that the branch of our algorithm described above is not rejected and the LP we obtain in this branch has a solution.

Now let us take any solution to the LP obtained in the branch described above and define the assignment π as in our algorithm. It remains to show that the matching instance admits a perfect matching. Since, it is a solution, it is easy to see that under this assignment the agents in A_V do not envy any other agent. Similarly, all the agents that we guessed in our branching do not envy any other piece. Note that all agents assigned the piece so far get the same "kind" of a piece as they get by π_{SOL} . It remains to show that if agent $a \in A$ gets a piece fully inside an edge $e \in E(G)$ according to π_{SOL} , then the auxiliary bipartite graph $H = (X \uplus Y, F)$ that our algorithm constructs contains an edge between a and all pieces fully inside e . That is, we need to check that assigning the agent a to a piece fully inside e does not cause envy. First note for every $a' \in A_V$, the agent a satisfies the inequality

$$\frac{\sum_{(f,i) \in \text{Pieces}_{a'}} u_{a_{e,a'}}(f) \cdot y_f^i}{u_{a_{e,a'}}(e)} \geq \frac{\sum_{(f,i) \in \text{Pieces}_{a'}} u_a(f) \cdot y_f^i}{u_a(e)},$$

by the choice of the agent $a_{e,a'}$ and hence the LP instance contains the constraint

$$u_a(e) \cdot \delta_e \geq \sum_{(f,i) \in \text{Pieces}_{a'}} u_a(f) \cdot x_f^i,$$

therefore, a does not envy any piece of an agent in A_V . Now, for an edge f , we know that

$$\frac{u_a(e)}{u_a(f)} \geq \frac{u_{a_{(e,f)}}(e)}{u_{a_{(e,f)}}(f)},$$

by the choice of the agent $a_{(e,f)}$, and that the LP instance contains a constraint

$$u_{a_{(e,f)}}(e) \cdot \delta_e \geq u_{a_{(e,f)}}(f) \cdot \delta_f.$$

Therefore,

$$u_a(e) \cdot \delta_e \geq u_a(f) \cdot \delta_f$$

and a would not envy a piece inside f . It follows that an agent a that is assigned a piece fully inside e by π_{SOL} can be assigned a piece fully inside e by π as well and thus H admits a perfect matching. $\square$

Lemma 14. *The runtime of the above algorithm is $|A|^{\mathcal{O}(k^2)}$.*

Proof. All operations such as adding a constraints in LP instance, checking the consistency of the instance, solving LP, creating the auxiliary graph H and finding maximum size matching in H can be done in polynomial time. Moreover, we can enumerate points in all cells of $\mathcal{P}$ over $\mathbb{R}^{2k}$ in $|A|^{\mathcal{O}(k)}$ time. Hence, to get running time $|A|^{\mathcal{O}(k^2)}$, it suffices to upper bound the number of branches by $|A|^{\mathcal{O}(k^2)}$. We first branch for each endpoint of an edge for the agent that gets this endpoints. This is $|A|^{2k}$ many branches. For each branch we then branch for each edge on the number of agents that are assigned a piece fully inside the edge. This is $(|A| + 1)^k$ many branches. Afterwards, we branch for each pair of edges e, f on the agent $a_{(e,f)}$. This is $|A|^{k^2}$ many branches. In each branch we run the algorithm from [Proposition 1](#) and branch on $|A|^{\mathcal{O}(k)}$ many points in $\mathbb{R}^{2k}$. This is $|A|^{\mathcal{O}(k)}$ many branches. Finally in each branch, we branch on agents $a_{e,a}$, this is at most $|A|^{2k^2}$ many branches (as $|A_V| \leq 2k$). Therefore, the total number of branches is

$$|A|^{2k} \cdot (|A| + 1)^k \cdot |A|^{k^2} \cdot |A|^{\mathcal{O}(k)} \cdot |A|^{2k^2} = |A|^{\mathcal{O}(k^2)}.$$

□

The proof of [Theorem 10](#) follows immediately from [Lemmas 12–14](#).

5. Concluding remarks

Our results provide a significantly improved understanding of the classical complexity of envy-free graph cutting. One direction that may be of interest for future work is to study approximate envy-freeness. After the publication of the conference version of this paper, Yuen and Suksompong [50] have initiated the study on this direction and have provided polynomial-time algorithms for this type of problems. While they have essentially completely resolved the case with identical valuations, there is plenty of room for improvement for the general case.

A different direction is to analyze the complexity of this problem through the more refined *parameterized complexity* paradigm. Indeed, in that setting the algorithmic results presented here can be viewed as XP algorithms. An immediate question in this context is whether these results can be strengthened to show fixed-parameter tractability. Most prominently, is there a fixed-parameter algorithm for EF-GC parameterized by the number of edges?

For the special case where the underlying graph is a path, the complexity of the problem is an even more intriguing question. As EF-GC on a path is a special case of EF CONTIGUOUS CAKE CUTTING, we know that it always admits a solution and hence the decision version of the problem cannot be W[1]-hard (for any parameterization). If the problem of computing an envy-free solution does not admit a fixed-parameter algorithm, would showing this require a variation of the W-hierarchy tailored specifically to TFNP problems [51]?

CRedit authorship contribution statement

Argyrios Deligkas: Writing – review & editing, Writing – original draft, Methodology, Funding acquisition, Formal analysis, Conceptualization; **Eduard Eiben:** Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization; **Robert Ganian:** Writing – review & editing, Writing – original draft, Methodology, Funding acquisition, Formal analysis, Conceptualization; **Thekla Hamm:** Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization; **Sebastian Ordyniak:** Writing – review & editing, Writing – original draft, Methodology, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Argyrios Deligkas reports financial support was provided by Engineering and Physical Sciences Research Council. Sebastian Ordyniak reports financial support was provided by Engineering and Physical Sciences Research Council. Robert Ganian reports financial support was provided by Austrian Science Fund. Thekla Hamm reports financial support was provided by Austrian Science Fund. Robert Ganian reports financial support was provided by Vienna Science and Technology Fund. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to thank the anonymous reviewers of ARTINT that helped us to significantly improve the exposition of our paper. Deligkas acknowledges the support by the EPSRC (EP/X039862/1). Ganian and Hamm acknowledge support from the [Austrian Science Fund](#) (FWF, Projects [P31336](#) and [Y1329](#)). Ganian acknowledges support from the [Austrian Science Fund](#) (FWF, projects [10.55776/Y1329](#) and [10.55776/COE12](#)) and the WWTF Science Fund (project ICT22-029). Hamm also acknowledges support from FWF (Project W1255-N23). Ordyniak acknowledges support by the EPSRC (EP/V00252X/1).

Data availability

No data was used for the research described in the article.

References

- [1] S.J. Brams, A.D. Taylor, *Fair Division: From Cake-Cutting to Dispute Resolution*, Cambridge University Press, 1996.
- [2] J. Robertson, W. Webb, *Cake-Cutting Algorithms: Be Fair If You Can*, CRC Press, 1998.
- [3] H. Moulin, *Fair Division and Collective Welfare*, MIT press, 2004.
- [4] A.D. Procaccia, Cake cutting: not just child's play, *Commun. ACM* 56 (7) (2013) 78–87.
- [5] L.E. Dubins, E.H. Spanier, How to cut a cake fairly, *Am. Math. Mon.* 68 (1P1) (1961) 1–17.
- [6] W. Stromquist, How to cut a cake fairly, *Am. Math. Mon.* 87 (8) (1980) 640–644.
- [7] X. Bei, E. Elkind, E. Segal-Halevi, W. Suksompong, Dividing a graphical cake, *SIAM J. Discrete Math.* 39 (1) (2025) 19–54.
- [8] X. Bei, A. Igarashi, X. Lu, W. Suksompong, Connected fair allocation of indivisible goods, (2019) arXiv preprint arXiv:1908.05433.
- [9] V. Bilò, I. Caragiannis, M. Flammini, A. Igarashi, G. Monaco, D. Peters, C. Vinci, W.S. Zwicker, Almost envy-free allocations with connected bundles, *Games Econ. Behav.* 131 (2021) 197–221.
- [10] S. Bouveret, K. Cechlárová, E. Elkind, A. Igarashi, D. Peters, Fair division of a graph, in: *IJCAI*, 2017, pp. 135–141.
- [11] A. Deligkas, E. Eiben, R. Ganian, T. Hamm, S. Ordyniak, The parameterized complexity of connected fair division, in: *IJCAI*, 2021, pp. 139–145.
- [12] E. Elkind, E. Segal-Halevi, W. Suksompong, Graphical cake cutting via maximin share, in: *IJCAI*, 2021, pp. 161–167.
- [13] A. Igarashi, D. Peters, Pareto-optimal allocation of indivisible goods with connectivity constraints, in: *AAAI*, 2019, pp. 2045–2052.
- [14] W. Suksompong, Fairly allocating contiguous blocks of indivisible items, *Discrete Appl. Math.* 260 (2019) 227–236.
- [15] A. Igarashi, W.S. Zwicker, Fair division of graphs and of tangled cakes, *Math. Program.* 203 (1) (2024) 931–975. <https://doi.org/10.1007/S10107-023-01945-5>
- [16] R. Abebe, J. Kleinberg, D.C. Parkes, Fair division via social comparison, in: *AAMAS*, 2017, p. 281–289.
- [17] H. Aziz, S. Bouveret, I. Caragiannis, I. Giagkousi, J. Lang, Knowledge, fairness, and social constraints, in: *AAAI*, 1, 2018.
- [18] X. Bei, Y. Qiao, S. Zhang, Networked fairness in cake cutting, in: *IJCAI*, 2017, pp. 3632–3638.
- [19] X. Bei, X. Sun, H. Wu, J. Zhang, Z. Zhang, W. Zi, Cake cutting on graphs: a discrete and bounded proportional protocol, in: *SODA*, 2020, pp. 2114–2123.
- [20] Y. Chevaleyre, U. Endriss, N. Maudet, Distributed fair allocation of indivisible goods, *Artif. Intell.* 242 (2017) 1–22.
- [21] E. Eiben, R. Ganian, T. Hamm, S. Ordyniak, Parameterized complexity of envy-free resource allocation in social networks, in: *AAAI*, 2020, pp. 7135–7142.
- [22] F.E. Su, Rental harmony: Sperner's lemma in fair division, *Am. Math. Mon.* 106 (10) (1999) 930–942.
- [23] X. Deng, Q. Qi, A. Saberi, Algorithmic solutions for envy-free cake cutting, *Oper. Res.* 60 (6) (2012) 1461–1476.
- [24] E. Segal-Halevi, W. Suksompong, How to cut a cake fairly: a generalization to groups, *Am. Math. Mon.* 128 (1) (2021) 79–83.
- [25] H. Hosseini, A. Igarashi, A. Searns, Fair division of time: multi-layered cake cutting, in: *IJCAI*, 2020, pp. 182–188.
- [26] A. Igarashi, F. Meunier, Envy-free division of multi-layered cakes, *Math. Oper. Res.* 50 (3) (2025) 2261–2286. <https://doi.org/10.1287/MOOR.2022.0350>
- [27] E.R. Arunachaleswaran, S. Barman, R. Kumar, N. Rathi, Fair and efficient cake division with connected pieces, in: *WINE*, 2019, pp. 57–70.
- [28] P. Goldberg, A. Hollender, W. Suksompong, Contiguous cake cutting: hardness results and approximation algorithms, *J. Artif. Intell. Res.* 69 (2020) 109–141.
- [29] M. Seddighin, M. Farhadi, M. Ghodsi, R. Alijani, A.S. Tajik, Expand the shares together: envy-free mechanisms with a small number of cuts, *Algorithmica* 81 (4) (2019) 1728–1755.
- [30] Y.J. Cohler, J.K. Lai, D.C. Parkes, A.D. Procaccia, Optimal envy-free cake cutting, in: *AAAI*, 2011.
- [31] D. Kurokawa, J.K. Lai, A.D. Procaccia, How to cut a cake before the party ends, in: *AAAI*, 2013.
- [32] R.J. Lipton, E. Markakis, E. Mossel, A. Saberi, On approximately fair allocations of indivisible goods, in: *EC*, 2004, pp. 125–131.
- [33] E. Elkind, E. Segal-Halevi, W. Suksompong, Mind the gap: cake cutting with separation, in: *AAAI*, 2021, pp. 5330–5338.
- [34] E. Elkind, E. Segal-Halevi, W. Suksompong, Keep your distance: land division with separation, in: *IJCAI*, 2021, pp. 168–174.
- [35] E. Segal-Halevi, A. Hassidim, Y. Aumann, Waste makes haste: bounded time algorithms for envy-free cake cutting with free disposal, *Trans. Algorithms* 13 (1) (2016) 1–32.
- [36] V. Menon, K. Larson, Deterministic, strategyproof, and fair cake cutting, in: *IJCAI*, 2017, pp. 352–358.
- [37] J. Marengo, T. Tetzlaff, Envy-free division of discrete cakes, *Discrete Appl. Math.* 164 (2014) 527–531.
- [38] I. Caragiannis, J.K. Lai, A.D. Procaccia, Towards more expressive cake cutting, in: *IJCAI*, 2011, pp. 127–132.
- [39] S. Brânzei, N. Nisan, Communication complexity of cake cutting, in: *EC*, 2019, p. 525.
- [40] S. Brânzei, P.B. Miltersen, A dictatorship theorem for cake cutting, in: *IJCAI*, 2015, pp. 482–488.
- [41] X. Bei, A. Igarashi, X. Lu, W. Suksompong, The price of connectivity in fair division, in: *AAAI*, 2021, pp. 5151–5158.
- [42] E. Balkanski, S. Brânzei, D. Kurokawa, A. Procaccia, Simultaneous cake cutting, in: *AAAI*, 2014, pp. 566–572.
- [43] Y. Aumann, Y. Dombb, The efficiency of fair division with connected pieces, *ACM TEAC* 3 (4) (2015) 1–16.
- [44] R. Diestel, *Graph Theory*, 4th Edition, Graduate texts in mathematics, Springer, 2012.
- [45] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, Springer-Verlag, 2006.
- [46] H.E. Warren, Lower bounds for approximation by nonlinear manifolds, *Trans. Am. Math. Soc.* 133 (1) (1968) 167–178.
- [47] K. Simonov, F.V. Pomin, P.A. Golovach, F. Panolan, Refined complexity of PCA with outliers, in: *ICML*, 2019, pp. 5818–5826.
- [48] J. Alman, V.V. Williams, A refined laser method and faster matrix multiplication, *TheoretCS* 3 (2024). <https://theoretcs.episciences.org/14213>
- [49] M.B. Cohen, Y.T. Lee, Z. Song, Solving linear programs in the current matrix multiplication time, in: *STOC*, 2019, pp. 938–942.
- [50] S.M. Yuen, W. Suksompong, Approximate envy-freeness in graphical cake cutting, *Discret. Appl. Math.* 357 (2024) 112–131. <https://doi.org/10.1016/J.DAM.2024.05.029>
- [51] N. Megiddo, C.H. Papadimitriou, On total functions, existence theorems and computational complexity, *Theor. Comput. Sci.* 81 (2) (1991) 317–324.
- [52] A. Deligkas, E. Eiben, R. Ganian, T. Hamm, S. Ordyniak, The complexity of envy-free graph cutting, *In Proc. of IJCA*, pp. 237–243. [ijcai.org](https://doi.org/10.1145/ijcai.2022), 2022.